Global Warming Science

https://courses.seas.harvard.edu/climate/eli/Courses/EPS101/

Forest Fires!

Please use the template below to answer the workshop questions. "XX" indicates places where you need to complete/write code or add a discussion.

Your name:

[ ]:
```
# Load libraries and data:
import numpy as np
import pickle
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import linregress
from scipy.signal import savgol_filter # for smoothing time series
import cartopy.crs as ccrs
import warnings

# load the data from a pickle file:
with open('./forest_fires_variables.pickle', 'rb') as file:
    while 1:
        try:
            d = pickle.load(file)
        except (EOFError):
            break
            # print information about each extracted variable:
        for key in d:
            print(
                f"extracting pickled variable: "
                f"name={key}; size={d[key].shape}"
            )
            # print(f"type={type(d[key])}")
        globals().update(d)
```

Explanation of input variables:

Canada fires:

canada_fire_area, canada_fire_years, canada_fire_number,

these represent area burnt in km^2 per year, and the number of fires per year, with corresponding time axes.

US fires:

US_fire_area, US_fire_years, US_fire_number,

these represent total US area burnt in km^2 per year, and the number of total US fires per year, with corresponding time axes.

global fires:

global annual burnt area time series in km^2 by type of burnt area. all fire area is the sum of the other components.

global_fire_years, global_crop_fire_area, global_forest_fire_area, global_savanna_fire_area, global_shrub_grass_fire_ar
global_all_fire_area,

Time series for extracting the ACC signal in forest area burnt over Western US:

west_US_VPD, west_US_VPD_NOACC (VPD without anthropogenic climate change), west_US_area_burnt (km^2/year)

VPD is in normalized dimensionless units, see notes.

Time series for analyzing the droughts that led to the 2019-2020 Australian fires:

australia_rain_Murray_Darling (mm/month), australia_rain_Murray_Darling_years, australia_IOD (deg C), australia_IOD_years,

australia_NINO34 (deg C), australia_NINO34_years, australia_SAM (non dimensional), australia_SAM_years

Ensemble time series for extracting ACC signal in surface temperature over the western US, separately from natural variability:

forest_fires_west_US_TS_ensemble_timeseries, forest_fires_west_US_TS_timeseries_ensemble_years

Surface temperature (K) time series averaged over the western US from multiple model ensembles from 1920 to 2000.

Data for calculating El Nino/La Nina SST composites:

SST_4composite, NINO34_timeseries_4composite, lat_4composite, lon_4composite, months_4composite, dates_4composite

1. Observed fire trends: (a) Plot the total burnt area and number of fires in the US as a function of year; (b) same in Candada; (c) burnt area in western US, (d) global burnt area, plotting each of the land types. Superimpose a regression line for each of the plots, and describe your findings.

[ ]:
```
plt.figure(figsize=(6,8),dpi=600)

# plot area burnt in US fires:
plt.subplot(4,1,1)
plt.plot(US_fire_years,US_fire_area/1.e3,'r')
plt.xlabel("year")
plt.ylabel("area burnt US ($10^3$ km$^2$)",color='r')
plt.grid(lw=0.25)
# calculate and plot regression:
x=US_fire_years; y=US_fire_area/1.e3
fit=linregress(x, y); slope=fit.slope; intercept=fit.intercept;
p=fit.pvalue; r2=(fit.rvalue)**2; fitted_line=intercept+slope*x
plt.plot(x,fitted_line,'r')
#plt.title("p=%g, r$^2$=%g" % (p,r2))

# add a plot of the number of US fires on same axes:
ax = plt.gca()
ax2 = ax.twinx()
ax2.plot(XX,XX,'b')
plt.xlabel("year")
ax2.set_ylabel("number of US fires",color='b')
# calculate and plot regression:
XX
```

```
ax2.plot(x,fitted_line,'b')
#plt.title("p=%g, r$^2$=%g" % (p,r2))


# plot area burnt in Canada fires:
plt.subplot(4,1,2)
plt.plot(XX,XX,'r')
plt.xlabel("year")
plt.ylabel("area burnt canada ($10^3$ km$^2$)")
plt.grid(lw=0.25)
# calculate and plot regression:
x=canada_fire_years; y=canada_fire_area/1.e3
fit=linregress(x, y); slope=XX; intercept=XX;
p=XX; r2=XX; fitted_line=XX+XX*x
plt.plot(x,fitted_line)
plt.title(f"p={p:g}, r$^2$={r2:g}")

# add number of Canada fires:
XX

# plot area burnt in western US fires:
plt.subplot(4,1,3)
plt.plot(west_US_years,west_US_area_burnt)
plt.xlabel("year")
plt.ylabel("west US burnt area")
plt.grid(lw=0.25)
# calculate and plot regression:
XX
plt.plot(x,fitted_line)
plt.title(f"p={p:g}, r$^2$={r2:g}")

# plot global fire area time series:
plt.subplot(4,1,4)
plt.plot(global_fire_years,global_forest_fire_area/1.e6,'r',label="Forest")
plt.plot(XX,XX,'g',label="Savanna")
plt.plot(XX,XX,'b',label="Shrub/grass")
plt.plot(XX,XX,'m',label="crop")
plt.plot(XX,XX,'k',label="All")
plt.xlabel("Year")
plt.ylabel("Area burnt ($10^6$ km$^2$)")
plt.legend(ncol=3)
plt.grid(lw=0.25)

# calculate and plot regression:
x=global_fire_years.flatten()
y=global_all_fire_area.flatten()/1.e6
fit=linregress(x, y); slope=fit.slope; intercept=fit.intercept;
p=fit.pvalue; r2=(fit.rvalue)**2; fitted_line=intercept+slope*x
plt.plot(x,fitted_line,'--k',linewidth=1)
plt.title("Global fire area; p=%g, r$^2$=%g" % (p,r2))

plt.tight_layout()
```

Discussion: XX

2. VPD as a fire danger index.

(a) Understanding VPD: Plot the saturation water vapor pressure and the water vapor partial pressure assuming an 80% relative humidity versus temperature. Plot the difference between the two curves (i.e., the VPD for a constant relative humidity) versus temperature.

[ ]:
```
# calculate *dimensional* VPD as function of temperature:
plt.figure(figsize=(7,3),dpi=300)

# plot moisture and saturation moisture vs temperature
plt.subplot(1,2,1)
Trange=np.asarray(np.arange(0,40,1))
# use Clausius-Clapeyron formula at a pressure of 1000 mb to calculate q_sat for␣
 ↪Trange:
Saturation_vapor_pressure=XX
# calculate the vapor pressure assuming relative humidity of 80%:
vapor_pressure_RH80=XX
# plot q^*(T) and q at RH=0.8:
plt.plot(Trange,XX,label="e^*(T)")
plt.plot(Trange,XX,label="e(T)=RH*e^*(T)")
plt.xlabel("Temperature")
plt.ylabel("water vapor pressure")

# plot VPD (difference between moisture and saturation moisture) vs temperature
# VPD for Trange and RH=80%:
VPD=XX
plt.subplot(1,2,2)
plt.plot(XX,XX)
plt.grid(lw=0.25)
plt.xlabel("T (C)")
plt.ylabel("VPD (mb)")

plt.tight_layout();
```

(b) Plot the western US VPD and area burnt versus time on the same axes. Then repeat using VPD and the log10 of area burnt. Calculate the correlation coefficient between the two plotted time series for each of the two cases.

[ ]:
```
# plot VPD given in input in nondimensional normalized units:

fig=plt.figure(figsize=(8,3),dpi=300)

plt.subplot(1,2,1)
# plot burnt area in 1000 of km^2 and VPD:
plt.plot(XX,XX,lw=1,color="r",label="area burnt")
ax=plt.gca();
plt.xlabel("XX")
plt.ylabel("area burnt (km$^2\\times1000$)",color="r")
plt.grid(lw=0.25)
# plot VPD on same axes:
ax1=plt.gca()
ax2 = ax1.twinx()
ax2.plot(XX,XX,color="b")
ax2.set_ylabel('XX', color='b')
print(
```

```
    f"correlation (VPD, area) = "
    f"{np.corrcoef(west_US_VPD, west_US_area_burnt)[0, 1]:g}"
)

plt.subplot(1,2,2)
# plot log10 burnt area in 1000 of km^2 and VPD:
XX
print("correlation (VPD,log10(area))=",XX)

plt.tight_layout();
```

3. Separating ACC from variability: Extract the ACC signal in temperature from an ensemble of model runs for 1920–2020. Plot all ensemble time series and superimpose the estimated ACC signal.

[ ]:
```
# calc avereage over all ensemble members:
TS_timeseries=1.0*forest_fires_west_US_TS_ensemble_timeseries
dates=1.0*forest_fires_west_US_TS_timeseries_ensemble_years
TS_timeseries_ensemble_avg=XX

# plot:
fig=plt.figure(dpi=300,figsize=(6,4))
plt.clf()
for ensemble_member in range(len(TS_timeseries[:,0])):
    # plot all ensemble averages with thin transparent lines (alpha parameter)
    plt.plot(XX,XX,lw=0.25,alpha=0.4)
TS_timeseries_avg_smooth = savgol_filter(XX) # check python docs for how to use filter
plt.plot(XX,XX,label="ensemble mean")
plt.plot(XX,XX,label="ensemble mean, smoothed")
plt.xlabel("year")
plt.ylabel("Surface Temperature")
plt.legend()

plt.tight_layout()
plt.show();
```

4. Estimate the contribution of ACC to western US forest fire area:

following Abatzoglou and Williams (2016):

(a) (i) Plot time series of long area burnt and VPD. (ii) Scatter plot area burnt vs. VPD, calculate and plot a regression line, calculate the r^2. (iii) Plot time series of the western US VPD, the VPD increase due to ACC, and the VPD without ACC. (iv) Using your calculated regression, calculate and plot the area that would have burned without ACC, and then the contribution of ACC as a function of time.

[ ]:
```
warnings.filterwarnings("ignore", message="invalid value encountered in less")

# calculate the effects of ACC on VPD:
west_US_VPD_ACC=west_US_VPD-west_US_VPD_NOACC
# Fit a regression line to the VPD-log10(burnt area) relation:
# -----------------------------------------------------
x=XX; y=XX;
```

```
fit=linregress(x, y)
slope=fit.slope;
intercept=fit.intercept;
p=fit.pvalue;
r2=(fit.rvalue)**2
print(f"p = {p:g}, r2 = {r2:g}")

# Calculate the effect of ACC VPD on fires using the calculated regression:
# ----------------------------------------------------------------------------
west_US_burnt_area_due_to_ACC=10**(XX)
# burnt area without effect of ACC:
west_US_area_burnt_NOACC=XX
# set meaningless negaative area to zero:
west_US_area_burnt_NOACC[west_US_area_burnt_NOACC<0]=0
years=west_US_years

fig=plt.figure(figsize=(5.6,4.5),dpi=300)

# --------------------------------
# plot burnt area in 1000 of km^2:
# --------------------------------
plt.subplot(2,2,1)
plt.plot(XX,XX,label="area burnt")
plt.plot(XX,XX,label="due to ACC")
plt.plot(XX,XX,label="no ACC")
plt.xlabel("year")
plt.ylabel("area burnt (km$^2\\times1000$)",color="r")
plt.legend()
plt.grid(lw=0.25)


# ---------------------------------------------
# scatter plot of VPD to burnt area regression:
# ---------------------------------------------
plt.subplot(2,2,2)
# scatter plot of VPD vs log10(area burnt):
plt.scatter(XX,XX)
# plot the fitted regression line:
plt.plot(XX,XX)
plt.xlabel("VPD",color="b")
plt.ylabel("log$_{10}$(area burnt)",color="r")

# ----------------
# plot VPD:
# ----------------
plt.subplot(2,2,3)
plt.plot(XX,XX,label="VPD")
plt.plot(XX,XX,label="due to ACC")
plt.plot(XX,XX,label="without ACC")
plt.xlabel("year")
plt.ylabel("VPD",color="b")
plt.legend()
plt.grid(lw=0.25)
```

```
# ----------------
# plot burnt area: in 1000 of km^2
# ----------------
plt.subplot(2,2,4)
plt.plot(years,XX,label="Area burnt")
plt.plot(years,XX,label="Due to ACC")
plt.plot(years,XX,label="No ACC")
plt.xlim([1983,2016])
ax=plt.gca();
plt.xlabel("Year")
plt.ylabel("Area burnt (km$^2\\times1000$)",color="r")


# finalize plot:
plt.tight_layout()
plt.show()
```

(b) What percent of area burnt over the last 15 years of data is attributable to ACC?

[ ]:
```
# total area burnt since 2000:
total_burnt=XX sum over area burnt over years after 2000
total_burnt_ACC=XX sum over area burnt due to ACC over years after 2000
print(f"total area burnt since 2000 = {total_burnt:g} km^2*1000")
print(f"total area burnt since 2000 due to ACC = {total_burnt_ACC:g} km^2*1000")
print(
    f"percent area burnt attributable to ACC = "
    f"{100 * total_burnt_ACC / total_burnt:g}%"
)
print("total forested area in the US according to Wikipedia: XX km^2*1000")
```

(c) Crudely estimate the change in normalized VPD due to a 2 °C warming by examining the provided time series of normalized VPD due to ACC and the western US temperature record in Figure 14.2. XX

(d) Given the linear dependence of log of the area burnt on normalized VPD, how much of the western US forest area (in both 10^3 km^2 and in percent of the total current forest area) might burn per year if the temperature increases by two more degree Celsius? XX

5. Role of variability modes: Calculate the averaged SAM/NINO3.4/IOD indices for rainy vs dry years (defined to be above and below one standard deviation, respectively) in south-east Australia, compare to the average over years 2018–9.

[this follows King et al. 2020]

[ ]:
```
# calculate std and mean of the four Australia time series (use np.mean and np.std):
rain_std=XX
rain_mean=XX
SAM_std=XX
SAM_mean=XX
IOD_std=XX
IOD_mean=XX
```

```
NINO34_std=XX
NINO34_mean=XX

print(f"rain_mean, rain_std = {rain_mean:g}, {rain_std:g}")
print(f"SAM_mean, SAM_std = {SAM_mean:g}, {SAM_std:g}")
print(f"IOD_mean, IOD_std = {IOD_mean:g}, {IOD_std:g}")
print(f"NINO34_mean, NINO34_std = {NINO34_mean:g}, {NINO34_std:g}")

print("\nrainy averages: averaged over rain > mean + std")

IOD_rainy_avg = np.mean(
    australia_IOD[australia_rain_Murray_Darling > rain_mean + rain_std]
)
print(f"IOD_rainy_avg = {IOD_rainy_avg:g}")

SAM_rainy_avg=XX
print(f"SAM_rainy_avg = {SAM_rainy_avg:g}")

NINO34_rainy_avg=XX
print(f"NINO34_rainy_avg = {NINO34_rainy_avg:g}")

print("\ndry averages: averaged over rain < mean - std")
XX

print("\naverages for 2018-2019:")
print(f"rain: {XX:g}")
print(f"SAM: {XX:g}")
print(f"IOD: {XX:g}")
print(f"NINO34: {XX:g}")
```

6. Optional extra credit: ENSO SST composites: Calculate El Niño and La Niña SST composites by removing the monthly averaged SST and then averaging over months in which the NINO3.4 time series is 1.5 std above or below its mean.

[ ]:
```
# ------------------------------------------------------------------------
# (1) prepare mask time series for calculating composites:
# El Nino composite mask: set to 1 if NINO34>mean+1.5*std, NaN otherwise
# La nina composite mask: set to 1 if NINO34<mean-1.5*std, NaN otherwise
# ------------------------------------------------------------------------


# ----------------------------------------------------------------
# (2) plot mask time series super imposed on NINO34 time series:
# ----------------------------------------------------------------


# ----------------------------------------------------------------
# (3) calculate monthly climatology of SST, remove it from SST data:
# ----------------------------------------------------------------


# ------------------------------------------------------------------------
# (4) use mask timeseries to calculate SST composites for El Nino, Lanina:
```

```
# ------------------------------------------------------------------------

# ------------------------------------------------
# (5) contour composites for El Nino and La Nina:
# ------------------------------------------------

# add a column to lon/lat arrays to eliminate white gap at dateline:
# for atmospheric plots:
lon1=1.0*lon[-1]+lon[2]-lon[1]; lon1=np.hstack((lon,lon1))

# initialize figure:
plt.clf();
projection=ccrs.PlateCarree(central_longitude=0.0);
fig=plt.figure(figsize=(9,6),dpi=300);
grid = plt.GridSpec(1, 2)#, wspace=0.4, hspace=0.3)
shrink=0.53

# El Nino:
# --------
axes=fig.add_subplot(1,2,1, projection=ccrs.PlateCarree(180))
axes.set_extent([-240, -60, -40, 40], crs=ccrs.PlateCarree())
axes.coastlines(resolution='110m',lw=0.3)
axes.stock_img()
plt.set_cmap('bwr')
DATA=SST_composite_ElNino
c=axes.pcolormesh(lon+179,lat-1, DATA,vmin=-3,vmax=3)
clb=fig.colorbar(c, shrink=shrink, pad=0.02,ax=axes)
#clb.set_label('°C')
axes.set_title('El Niño')


# La Nina:
# --------
axes=fig.add_subplot(1,2,2, projection=ccrs.PlateCarree(180))#grid[0, 0])
axes.set_extent([-240, -60, -40, 40], crs=ccrs.PlateCarree())
axes.coastlines(resolution='110m',lw=0.3)
axes.stock_img()
plt.set_cmap('bwr')
DATA=1.0*SST_composite_LaNina
c=axes.pcolormesh(lon+179,lat-1, DATA,vmin=-3,vmax=3)
clb=plt.colorbar(c, shrink=shrink, pad=0.02,ax=axes)
#clb.set_label('°C')
axes.set_title('La Niña')

# finalize and show plot:
plt.subplots_adjust(top=0.92, bottom=0.08, left=0.01, right=0.95 \
                    ,hspace=-0.3,wspace=-0.01);
plt.show();
```

[ ]: